

La logica dietro la struttura heap

L'algoritmo usa la struttura dati *heap* per poter estrarre rapidamente il valore massimo ($\Theta(\log n)$) sfruttando la logica del *max heap*, ovvero la *chiave* di un qualsiasi nodo v sarà sempre maggiore della chiave dei suoi figli.

Un heap è un **albero binario** completo, quindi ogni nodo avrà due figli, fino al penultimo livello, ed è una struttura dati rappresentata tramite **vettore di posizione**, quindi preso un qualsiasi nodo v per trovare i suoi figli seguiremo la solita formula $P[d \cdot v + i]$, che possiamo specificare come $P[2 \cdot v]$ per il figlio destro e $P[2 \cdot v + 1]$ per il sinistro.

L'algoritmo

Algorithm 1: heapsort

```
1 heapsort (array  $A$ )
2   buildMaxHeap( $A$ );
   // estrazione dei valori dall'heap
3   for  $i = n$  to 1 do
4     swap  $A[i], A[1]$ ;
5      $n \leftarrow n - 1$ ;
6     heapify( $A, 1$ );
```

L'algoritmo presenta due funzioni ausiliarie importantissime che andremo ad analizzare nel dettaglio

Algorithm 2: buildMaxHeap

```
1 buildMaxHeap (array  $A$ )
2    $n \leftarrow A.len()$ ;
   // costruzione dell'heap solo sui nodi padri per
   // salvarci del tempo
3   for  $i = \lfloor n/2 \rfloor$  to 1 do
4     heapify( $A, i$ );
```

buildMaxHeap si occupa della costruzione della struttura dati dal semplice array iniziale. Notare come non siamo iterando su ogni nodo ma soltanto sulla prima metà perchè la funzione successiva, *heapify*, considera ogni figlio successivo.

Algorithm 3: heapify

```
1 heapify (array A, int father)
2   newFather  $\leftarrow$  father;
3   left  $\leftarrow$  father  $\times$  2;
4   right  $\leftarrow$  father  $\times$  2 + 1;

   // se l'indice left ha "sforato" la lunghezza dell'array
   // e se è maggiore del padre
5   if left  $\leq$  A.len() and A[left] > A[father] then
6     newFather  $\leftarrow$  left;

   // se l'indice right ha "sforato" la lunghezza dell'array
   // e se è maggiore del padre
7   if right  $\leq$  A.len() and A[right] > A[father] then
8     newFather  $\leftarrow$  right;

   // se il padre non è cambiato
9   if newFather  $\neq$  father then
10    swap A[father], A[newFather];
11    heapify(A, newFather);
```

Come già detto prima l'algoritmo sfrutta la formula $P[2 \cdot v]$ e $P[2 \cdot v + 1]$ per trovare subito i figli del padre su cui stiamo attualmente lavorando. L'array su cui stiamo lavorando non è ovviamente ordinato quindi i due *if* servono proprio per gestire l'ordine tra i padri ed i figli.

Costo computazionale

Il costo verrà ovviamente dettato dalla somma delle due funzioni, quindi:

$$T(n) = \text{BuildMaxHeap} + \text{Heapify in Heapsort}$$

Come detto prima *buildMaxHeap* itera usando un for a "metà", ma nel caso peggiore quell'altra metà sarà analizzata da Heapify, portando il costo a $O(n)$. Infine notiamo come la funzione originale *heapsort* itera usando un for da n a 1, ma non esegue esattamente n cicli poichè ad ogni iterazione viene decrementato $i = i - 1$, ma succede lo stesso con $n = n - 1$, portando quindi ad avere $\log n$ cicli, in cui al suo interno avremo un *heapify* con costo $O(n)$.

$$T(n) = \text{BuildMaxHeap} + \text{Heapify in Heapsort}$$

$\downarrow$

$$T(n) = O(n) + O(n \cdot \log n) = O(n \cdot \log n)$$